

SOFTWARE ENGINEERING · COURSE STUDIO

AI-assisted engineering. You are still the pilot.

Use AI as a thoughtful pair programmer—while you remain responsible for the problem, the proof, and the release.

PLAN CLEARLY · REVIEW CLOSELY · SHIP WITH EVIDENCE



AI Is a Teammate, Not an Autopilot

What AI Does Well

- Explaining unfamiliar code or syntax
- Generating boilerplate and test candidates
- Suggesting alternative approaches
- Spotting syntax errors quickly

Where AI Fails

- Understanding complex business logic
- Knowing your specific system architecture
- Ensuring security and privacy compliance
- Making final architectural decisions

Human Accountability

You are the engineer. You own the code, the bugs, and the outcomes. The AI is just a tool to help you get there faster.

"The pilot flies the plane. The autopilot only assists when conditions are known and safe."

Use a Reliable Engineering Loop

01 Brief

Define the goal, constraints, and context clearly before writing any code.

02 Ask

Prompt the AI with specific, bounded instructions and a definition of done.

03 Inspect

Read and understand every single line of suggested code. Do not blindly copy.

04 Test

Verify behavior with unit tests and manual checks to ensure correctness.

05 Review

Check for edge cases, security flaws, and style consistency before shipping.



Prompt With Context and a Definition of Done

Weak Prompt

```
"Write a function to fetch user data."
```

Strong Engineering Prompt

```
"Create an async function  
fetchUserProfile(userId: string) that calls  
our /api/v1/users endpoint."
```

Context: We use fetch API and standard try/catch.

Constraints: Do not use external libraries like axios.

Done criteria: Returns the parsed JSON on success, throws a custom Error('User not found') on 404, and logs other errors to console."

The Anatomy of a Good Prompt

1 Goal

What are you trying to achieve? Be specific about the exact function or component.

2 Context

Provide relevant files, tech stack details, and existing patterns in your codebase.

3 Constraints

Explicitly state what the AI should NOT do (e.g., forbidden libraries, anti-patterns).

4 Definition of Done

How will we know it works? Define success criteria, error handling, and edge cases.

Ask AI to Explain Before You Change

Before modifying an unfamiliar codebase, use AI to build a mental map. It is an excellent tool for reading and summarizing complex logic.

Effective Exploration Prompts

- | | |
|--|---|
| → "Explain how data flows from the API to this component." | → "Where is state managed for the user session?" |
| → "What does this regex actually do?" | → "Summarize the responsibilities of this class." |

```
function processData(input) {  
  const result = input.filter(x => x.active)  
    .map(x => ({ ...x, score: x.val * 2 }))  
    .reduce((acc, curr) => { acc[curr.id] = curr; return acc; }, {});  
  return Object.values(result).sort((a, b) => b.score - a.score);  
}
```

AI EXPLANATION

This function filters for active items, doubles their 'val' into a 'score', deduplicates by 'id', and returns an array sorted by score descending.

Implement Small, Reviewable Changes

AI can generate hundreds of lines of code in seconds. If you ask it to build an entire feature at once, you won't be able to properly review it.

The Vertical Slice Approach

1. Ask for the data model first. Review it.
2. Ask for the API fetch logic. Review it.
3. Ask for the basic UI component. Review it.
4. Ask to wire them together. Review it.

HUGE DIFF (HARD TO REVIEW)

+ 452 additions

- 128 deletions

Too much context to hold in your head. Bugs hide easily in large AI-generated blocks.

SMALL DIFF (EASY TO REVIEW)

+ 45 additions

- 12 deletions

Focused on a single concern. Easy to verify against requirements and test.

Debug With Evidence, Not Guesses

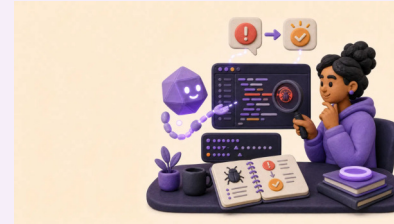
AI cannot read your mind or see your screen — say what you observed and include exact errors or logs.

THE GUESSING GAME

"My fetch function isn't working. Fix it."

THE EVIDENCE-BASED APPROACH

```
"TypeError: Cannot read properties of undefined  
(reading 'map') on line 42."
```



- **Stack Traces:** Exact error and line.
- **Console Logs:** What the data looks like.
- **Expected vs Actual:** What you wanted vs what happened.

Generate Tests, Then Verify Assertions

AI can generate many unit tests quickly, but strong assertions are still your responsibility.

The Happy Path Trap

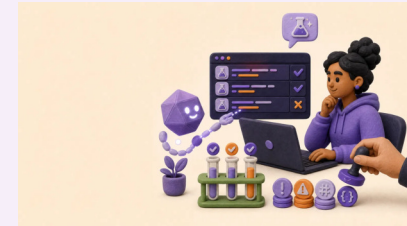
AI often tests only the obvious success case; explicitly ask for edge cases and failures.

The Tautology Trap

Avoid tests that merely mirror implementation logic (e.g., expecting x to equal x).

Verify the Business Logic

Confirm `expect()` statements reflect your actual rules and exceptions.



HUMAN VERIFICATION REQUIRED

```
// AI guesses a standard 10% discount:  
expect(getDiscount(100)).toBe(10);  
  
// Verify the actual rule:  
expect(getDiscount(100)).toBe(0);
```


Refactor for Clarity, Keep Behavior Safe

The Safe Refactoring Workflow

1 Lock Behavior with Tests

Never refactor without passing tests — they are your safety net.

2 Prompt for Clarity

Refactor to descriptive names and extract validation; keep return values.

3 Verify the Diff

Ensure only structure and naming changed, not logic.

4 Run Tests Again

If tests remain green, the refactor is safe to commit.

BEFORE (MESSY & UNCLEAR)

```
function process(x,y){  
  let a=x*0.2;  
  if(y)a+=10;  
  returnx+a;  
}
```

AFTER AI REFACTOR (CLEAR & MAINTAINABLE)

```
// AI renamed variables to reveal intent  
function calculateTotal(basePrice,hasShipping)  
{  
  let taxAmount=basePrice*0.2;  
  if(hasShipping)taxAmount+=10;  
  returnbasePrice+taxAmount;  
}
```

Protect Secrets and Review Risk

AI models are powerful, but they lack context on your security requirements and may inadvertently introduce risks into your codebase.

1. PROTECT SECRETS

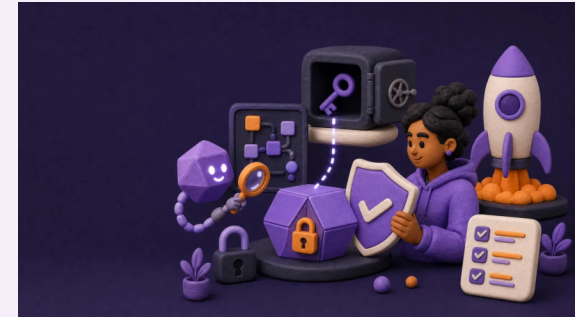
Never paste API keys, database credentials, or customer PII into an AI prompt. Always use environment variables.

2. VERIFY DEPENDENCIES

AI often hallucinates library names or suggests outdated, vulnerable packages. Always check the official registry.

3. AUDIT FOR VULNERABILITIES

AI can generate code susceptible to SQL injection or XSS. Explicitly prompt for security and review the output.



RULE OF THUMB

If you wouldn't post the code or data on a public forum, do not paste it into an AI prompt.

Mini Review: Spot the Assumption

DEVELOPER PROMPT

"My login page is returning a 500 Internal Server Error. How do I fix it?"

AI SUGGESTION

"A 500 error usually means your JWT token is expired. You need to change your token expiration time in the config file to 30 days. Here is the updated configuration code..."

What is the danger of accepting this suggestion?

- **Zero** A 500 error is a generic server crash. It
Evidence: could be a database timeout, a syntax error, or a missing environment variable. The AI is purely guessing.
- **Security** Extending a token expiration to 30 days is
Risk: a massive security vulnerability, suggested casually as a "fix."
- **The** Never change configuration based on a guess.
Fix: Check the server logs for the actual stack trace first.

Studio Lab: Ship a Verified Feature

Your turn to be the pilot. Use an AI assistant to build a new feature, but prove you are in control of the logic, security, and quality.

DELIVERABLES

- **Working Code:** A new API endpoint or UI component built with AI assistance.
- **Test Suite:** Passing unit tests that verify the core business logic.
- **Audit Trail:** A Markdown file logging your prompts, the AI's mistakes, and how you corrected them.

SUCCESS CRITERIA

- **Zero Secrets:** No API keys or PII were included in your prompt history.
- **Evidence-Based:** Debugging prompts include stack traces and logs, not just "it's broken."
- **Human Readability:** The final code is refactored for clarity and uses descriptive naming.

EXTENSION: THE HALLUCINATION TRAP

Intentionally write a vague prompt for a complex logic problem. Document the AI's incorrect guess, then write the corrected, evidence-based prompt that guides it to the right solution.